

Spring Boot - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

SPRING BOOT — MASTER NOTES	7
A Complete Reference Book for Interviews, Production Work & System Understanding	7
TOPIC 1: AUTO CONFIGURATION	7
1. Introduction	7
1.1 What is it?	7
1.2 Why was it introduced — The EXACT Problem It Solves	7
1.3 How Auto Configuration Actually Works Internally	8
1.4 Interview Definition	8
2. Real Life Analogy	8
3. Seeing Auto Configuration in Action — Debugging Tool	9
4. Overriding Auto Configuration	9
5. Common Mistakes	9
6. Frequently Asked Interview Questions	9
TOPIC 2: STARTER DEPENDENCIES	9
1. Introduction	10
2. Common Starters — Quick Reference	10
3. Why Starters Matter — Version Management (BOM)	10
4. Real Life Analogy	10
5. Frequently Asked Interview Questions	10
TOPIC 3: REST APIS	11
1. Introduction	11
2. Building a Complete CRUD REST Controller	12
3. Key Annotations — Full Reference	13
4. Real Life Analogy	13
5. Edge Cases	13
6. Common Mistakes	13
7. Best Practices	14
8. Frequently Asked Interview Questions	14

TOPIC 4: VALIDATION 14

1. Introduction	14
2. Code Example — Validated DTO + Controller	15
3. Key Validation Annotations	15
4. Handling Validation Errors Gracefully	16
5. Real Life Analogy	16
6. Edge Cases	16
7. Common Mistakes	16
8. Frequently Asked Interview Questions	16

TOPIC 5: CONFIGURATION PROPERTIES 17

1. Introduction	17
2. application.properties vs application.yml	17
3. Reading Individual Values — @Value	17
4. Reading GROUPED Settings — @ConfigurationProperties (The Recommended, Type-Safe Way)17	
5. Real Life Analogy	18
6. Edge Cases	18
7. Best Practices	18
8. Frequently Asked Interview Questions	18

TOPIC 6: PROFILES 19

1. Introduction	19
2. Profile-Specific Property Files	19
3. Activating a Profile	19
4. Profile-Specific Beans in Code	20
5. Real Life Analogy	20
6. Common Mistakes	20
7. Frequently Asked Interview Questions	20

TOPIC 7: LOGGING 20

1. Introduction	21
2. Why NOT System.out.println() in Production Code	21

3. Code Example — Using SLF4J	21
4. Configuring Log Levels	22
5. The Five Log Levels (Increasing Severity)	22
6. Real Life Analogy	22
7. Common Mistakes	22
8. Frequently Asked Interview Questions	22

TOPIC 8: EXCEPTION HANDLING **23**

1. Introduction	23
2. Code Example — A Global Exception Handler	23
3. How Controllers Become Dramatically Cleaner	24
4. Real Life Analogy	24
5. Edge Cases	24
6. Best Practices	24
7. Frequently Asked Interview Questions	24

TOPIC 9: SCHEDULING **25**

1. Introduction	25
2. Enabling Scheduling	25
3. Code Examples — The Three Scheduling Styles	25
4. fixedRate vs fixedDelay — A Frequently Confused Pair	26
5. Cron Expression Format	26
6. Real Life Analogy	26
7. Edge Cases	26
8. Common Mistakes	26
9. Frequently Asked Interview Questions	26

TOPIC 10: ASYNC PROGRAMMING **27**

1. Introduction	27
2. Enabling and Using @Async	27
3. Getting a Return Value from an Async Method — CompletableFuture	28
4. Real Life Analogy	28
5. Edge Cases	28

6. Common Mistakes	28
7. Frequently Asked Interview Questions	28

TOPIC 11: FILE UPLOAD **29**

1. Introduction	29
2. Code Example — A File Upload Endpoint	29
3. Configuring Upload Size Limits	30
4. MultipartFile Key Methods	30
5. Real Life Analogy	30
6. Edge Cases	30
7. Common Mistakes	30
8. Best Practices	30
9. Frequently Asked Interview Questions	31

TOPIC 12: EMAIL INTEGRATION **31**

1. Introduction	31
2. Configuration	31
3. Code Example — Sending a Simple Email	32
4. HTML Emails with Attachments (MimeMessage)	32
5. Real Life Analogy	32
6. Common Mistakes	32
7. Frequently Asked Interview Questions	33

TOPIC 13: SWAGGER/OPENAPI **33**

1. Introduction	33
2. Why It Matters — The Problem It Solves	33
3. Setup	33
4. Enhancing Documentation with Annotations	34
5. Real Life Analogy	34
6. Common Mistakes	34
7. Frequently Asked Interview Questions	34
8. Revision Notes (Entire Spring Boot Section Consolidated)	35
9. Homework (Covers Entire Spring Boot Section)	35

10. Mini Project — A Complete Employee Management REST API	36
FINAL SUMMARY — SPRING BOOT COMPLETE	36

SPRING BOOT — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: AUTO CONFIGURATION

1. Introduction

1.1 What is it?

Auto Configuration is Spring Boot's signature feature — it AUTOMATICALLY configures Spring beans (a `DataSource`, a Jackson JSON mapper, an embedded Tomcat server, etc.) based purely on WHAT'S ON YOUR CLASSPATH and simple property settings, WITHOUT you writing the explicit `@Bean` configuration code that "plain" Spring (Spring Core, previous section) would have required.

1.2 Why was it introduced — The EXACT Problem It Solves

Before Spring Boot (2014), setting up EVEN A SIMPLE Spring web application required substantial manual configuration:

```
// THE "BEFORE SPRING BOOT" WORLD - manually configuring a DataSource, JPA, and an embedded server
@Configuration
@EnableWebMvc
@EnableTransactionManagement
public class AppConfig {

    @Bean
    public DataSource dataSource() {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:postgresql://localhost:5432/company_db");
        config.setUsername("postgres");
        config.setPassword("password");
        return new HikariDataSource(config);
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory() {
        // ... 15+ more lines configuring Hibernate/JPA properties manually ...
    }

    @Bean
    public PlatformTransactionManager transactionManager() {
        // ... more manual wiring ...
    }

    // ... PLUS manually configuring an embedded Tomcat server, Jackson JSON settings, etc. ...
}
```

This was:

- **Extremely repetitive** — nearly EVERY Spring web project needed the SAME boilerplate `DataSource`/JPA/server configuration.

- **Error-prone** — easy to misconfigure a property, especially for beginners.
- **Slow to get started** — even a trivial "Hello World" REST API required substantial setup BEFORE writing any actual business logic.

With Spring Boot's Auto Configuration, the SAME setup becomes:

```
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}

# application.properties - just a few simple lines
spring.datasource.url=jdbc:postgresql://localhost:5432/company_db
spring.datasource.username=postgres
spring.datasource.password=password
```

Spring Boot AUTOMATICALLY detects (because you added the right dependency JARs, covered in Topic 2) that you want a DataSource, JPA/Hibernate, and an embedded Tomcat server — and configures ALL of it correctly, with sensible defaults, based on that classpath detection PLUS your simple property values.

1.3 How Auto Configuration Actually Works Internally

1. Spring Boot scans the CLASSPATH at startup (what JAR dependencies are present)
2. It checks CONDITIONS - e.g., "IS a DataSource class present on the classpath? IS there NO DataSource bean ALREADY manually defined by the developer?"
3. IF the conditions are met, Spring Boot AUTOMATICALLY registers a sensible DEFAULT bean configuration for that feature (e.g., a HikariCP DataSource, using your application.properties values)
4. IF you EXPLICITLY define your OWN @Bean for something (e.g., your own DataSource), Spring Boot's auto-configuration BACKS OFF and uses YOUR bean instead

Under the hood, this is implemented via annotations like `@ConditionalOnClass` (only auto-configure if a certain class is on the classpath) and `@ConditionalOnMissingBean` (only auto-configure if the developer hasn't ALREADY defined their own bean) — a real, inspectable Java mechanism, not "magic."

1.4 Interview Definition

"Auto Configuration is Spring Boot's mechanism for automatically configuring beans based on classpath contents and simple property settings, using conditional annotations (`@ConditionalOnClass`, `@ConditionalOnMissingBean`) to apply sensible defaults while still allowing developers to override any auto-configured bean with their own explicit definition."

2. Real Life Analogy

- **Restaurant:** "Plain" Spring is like being handed a completely empty commercial kitchen SHELL — you must personally install and wire up EVERY single stove, sink, and refrigeration unit yourself before cooking anything. Spring Boot's Auto Configuration is like moving into a FULLY PRE-EQUIPPED kitchen that automatically detects what equipment YOU actually brought (your dependencies) and instantly connects/configures it correctly — while still letting you swap out any SPECIFIC piece of equipment yourself if you have particular requirements.

3. Seeing Auto Configuration in Action — Debugging Tool

```
# application.properties - shows EXACTLY which auto-configurations were applied and why
debug=true
```

Running your application with this produces a detailed report showing which auto-configuration classes **MATCHED** (and were applied) versus which were **EXCLUDED** (and exactly why — e.g., "DataSourceAutoConfiguration did not match: required class javax.sql.DataSource not found").

4. Overriding Auto Configuration

```
@Configuration
public class CustomDataSourceConfig {
    @Bean
    public DataSource dataSource() {
        // Your OWN custom DataSource setup - Spring Boot's auto-configuration
        // AUTOMATICALLY backs off and uses THIS bean instead, since @ConditionalOnMissingBean
        // detects you've already provided one
        return new CustomPooledDataSource();
    }
}
```

5. Common Mistakes

- Assuming Auto Configuration is unexplainable "magic" — it's built on inspectable, standard Spring conditional annotations; the `debug=true` property lets you see EXACTLY what happened and why.
- Not realizing that defining your OWN `@Bean` for something Spring Boot would have auto-configured causes it to gracefully back off — a deliberate, useful design, not a conflict/error.

6. Frequently Asked Interview Questions

- 1 **What is Spring Boot's Auto Configuration?** A mechanism that automatically configures beans (DataSource, embedded server, JSON mapper, etc.) based on classpath contents and simple property values, eliminating the manual `@Bean` boilerplate "plain" Spring required.
- 2 **What annotation-based mechanism powers Auto Configuration internally?** Conditional annotations like `@ConditionalOnClass` (checks classpath presence) and `@ConditionalOnMissingBean` (checks whether the developer already defined their own bean).
- 3 **What happens if you define your own `@Bean` for something Spring Boot would normally auto-configure?** Spring Boot's auto-configuration gracefully backs off and uses YOUR bean instead — a deliberate, intentional override mechanism.
- 4 **How can you see exactly which auto-configurations were applied (and why others weren't)?** Set `debug=true` in `application.properties` and review the detailed auto-configuration report printed at startup.

TOPIC 2: STARTER DEPENDENCIES

1. Introduction

What is it? A "Starter" is a single Maven/Gradle dependency that bundles together ALL the individual libraries commonly needed for a specific type of functionality — e.g., `spring-boot-starter-web` pulls in Spring MVC, an embedded Tomcat server, and Jackson (for JSON) ALL AT ONCE, instead of you needing to identify and manually add each individual JAR (and worry about VERSION compatibility between them) yourself.

2. Common Starters — Quick Reference

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId> <!-- REST APIs, embedded Tomcat, Jackson JSON -->
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId> <!-- Spring Data JPA, Hibernate -->
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId> <!-- Spring Security -->
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-validation</artifactId> <!-- Bean Validation (covered in Topic 4) -->
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId> <!-- JUnit, Mockito, Spring Test utilities -->
<scope>test</scope>
</dependency>
```

3. Why Starters Matter — Version Management (BOM)

Spring Boot manages a "Bill of Materials" (BOM) that specifies EXACTLY which COMPATIBLE version of every underlying library (Hibernate, Jackson, Tomcat) to use TOGETHER — so when you add `spring-boot-starter-data-jpa`, you get a Hibernate version GUARANTEED to work correctly with your Spring Boot version, without you needing to manually research/pin compatible version numbers yourself.

4. Real Life Analogy

- **Restaurant:** A Starter is like ordering a pre-assembled "Full Breakfast Combo" from a menu, instead of individually selecting and separately confirming the exact compatible pairing of eggs, bacon, toast, and coffee yourself — the combo GUARANTEES everything works together correctly, in one simple order.

5. Frequently Asked Interview Questions

- 1 **What is a Spring Boot "Starter"?** A single dependency that bundles together all the commonly-needed libraries for a specific feature area (web, JPA, security), avoiding manual individual dependency management.

- 2 **What problem does the Spring Boot BOM (Bill of Materials) solve?** It guarantees COMPATIBLE versions of all underlying libraries are used together, eliminating manual version-compatibility research/pinning.
 - 3 **Name the starter you'd add for building REST APIs.** `spring-boot-starter-web`.
-

TOPIC 3: REST APIS

1. Introduction

What is it? This topic covers building actual REST APIs using Spring Boot's `@RestController` — the practical, hands-on core of most real Spring Boot job work, tying together everything from Spring Core (beans, DI) with web-specific annotations for handling HTTP requests/responses.

(Full REST architectural principles — HTTP methods, status codes, DTOs, versioning — are covered in depth in the dedicated REST APIs section later in this course. This topic focuses specifically on the Spring Boot IMPLEMENTATION mechanics.)

2. Building a Complete CRUD REST Controller

```
package com.jobsradar.employeeapi;

import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;
import org.springframework.http.HttpStatus;
import java.util.List;

@RestController // = @Controller + @ResponseBody (returns data, not a view)
@RequestMapping("/api/employees") // BASE path for every method in this controller
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) { // constructor injection (Spring Core,
        Topic 2)
        this.employeeService = employeeService;
    }

    @GetMapping // GET /api/employees
    public List<Employee> getAllEmployees() {
        return employeeService.findAll();
    }

    @GetMapping("/{id}") // GET /api/employees/5
    public ResponseEntity<Employee> getEmployee(@PathVariable Long id) {
        Employee emp = employeeService.findById(id);
        if (emp == null) {
            return ResponseEntity.notFound().build(); // returns HTTP 404
        }
        return ResponseEntity.ok(emp); // returns HTTP 200 + the employee JSON
    }

    @PostMapping // POST /api/employees
    @ResponseStatus(HttpStatus.CREATED) // returns HTTP 201 on success
    public Employee createEmployee(@RequestBody Employee employee) {
        return employeeService.save(employee);
    }

    @PutMapping("/{id}") // PUT /api/employees/5
    public ResponseEntity<Employee> updateEmployee(@PathVariable Long id, @RequestBody Employee employee)
    {
        Employee updated = employeeService.update(id, employee);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}") // DELETE /api/employees/5
    public ResponseEntity<Void> deleteEmployee(@PathVariable Long id) {
        employeeService.delete(id);
        return ResponseEntity.noContent().build(); // returns HTTP 204 (success, no body)
    }

    @GetMapping("/search") // GET /api/employees/search?dept=Engineering
    public List<Employee> searchByDepartment(@RequestParam String dept) {
        return employeeService.findByDepartment(dept);
    }
}
```

3. Key Annotations — Full Reference

Annotation	Purpose
<code>@RestController</code>	Marks a class as a REST controller — every method's return value is automatically serialized to JSON (via Jackson) and written directly to the HTTP response body
<code>@RequestMapping</code>	Sets a BASE URL path for the entire controller
<code>@GetMapping/@PostMapping/@PutMapping/@DeleteMapping</code>	Shorthand for <code>@RequestMapping(method = ...)</code> , mapping specific HTTP methods to specific handler methods
<code>@PathVariable</code>	Extracts a value from the URL PATH itself (e.g., the 5 in <code>/employees/5</code>)
<code>@RequestParam</code>	Extracts a value from the QUERY STRING (e.g., <code>?dept=Engineering</code>)
<code>@RequestBody</code>	Deserializes the incoming HTTP REQUEST BODY (typically JSON) into a Java object automatically
<code>ResponseEntity<T></code>	Gives full control over the HTTP status code, headers, AND body of the response

4. Real Life Analogy

- **Restaurant:** A `@RestController` is like a dedicated ORDER-TAKING STATION that ALWAYS communicates in a single, standardized format (JSON) — `@PathVariable` is like reading the specific TABLE NUMBER written on an order slip, `@RequestParam` is like reading OPTIONAL special instructions attached separately, and `@RequestBody` is like receiving a FULL, structured order form all at once to be processed.

5. Edge Cases

Case	Result
<code>@RequestBody</code> JSON missing a required field the target Java class expects	Typically deserializes successfully with that field as <code>null</code> (unless validation, Topic 4, is applied) — NOT an automatic error by itself
Sending malformed JSON in the request body	<code>HttpMessageNotReadableException</code> — typically results in an HTTP 400 Bad Request response
A <code>@PathVariable</code> type mismatch (e.g., <code>/employees/abc</code> when <code>id</code> is declared as <code>Long</code>)	<code>MethodArgumentTypeMismatchException</code> — typically an HTTP 400 response

6. Common Mistakes

- Returning raw `@Entity` objects DIRECTLY from a REST controller (rather than a proper DTO) — risks `LazyInitializationException` on lazy Hibernate relationships (see the Hibernate section's Lazy vs Eager deep dive) AND exposes your internal database structure directly to API consumers.
- Forgetting `@ResponseStatus/ResponseEntity` and defaulting to HTTP 200 for EVERYTHING, even genuine creation (201) or deletion (204) scenarios.

7. Best Practices

- Map `@Entity` objects to separate DTOs before returning them from a controller — avoids lazy-loading issues and decouples your API's public contract from your internal database schema.
- Use `ResponseEntity` (or `@ResponseStatus`) to return semantically correct HTTP status codes, not just always `200`.

8. Frequently Asked Interview Questions

- 1 **What does `@RestController` do differently from `@Controller`?** It automatically serializes method return values directly to the HTTP response body (typically as JSON) rather than resolving a view template to render.
 - 2 **What's the difference between `@PathVariable` and `@RequestParam`?** `@PathVariable` extracts a value from the URL PATH itself; `@RequestParam` extracts a value from the QUERY STRING.
 - 3 **What does `@RequestBody` do?** Automatically deserializes the incoming HTTP request body (typically JSON) into a Java object.
 - 4 **Why is it risky to return `@Entity` objects directly from a REST controller?** It can trigger `LazyInitializationException` on unfetched lazy relationships, and unnecessarily exposes internal database structure as your public API contract — mapping to a DTO avoids both issues.
-

TOPIC 4: VALIDATION

1. Introduction

What is it? Bean Validation (`jakarta.validation`, implemented by Hibernate Validator) lets you declare VALIDATION RULES directly on your DTO/entity fields via annotations, and Spring automatically enforces them on incoming `@RequestBody` data BEFORE your controller logic even runs — eliminating repetitive, manual `if (field == null) throw ...` checks scattered throughout your code.

2. Code Example — Validated DTO + Controller

```
import jakarta.validation.constraints.*;

public class EmployeeDto {

    @NotBlank(message = "First name is required")
    private String firstName;

    @NotNull(message = "Salary is required")
    @Positive(message = "Salary must be positive")
    private Double salary;

    @Email(message = "Invalid email format")
    private String email;

    @Min(value = 18, message = "Age must be at least 18")
    @Max(value = 65, message = "Age must be at most 65")
    private Integer age;

    // getters and setters...
}

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @PostMapping
    public ResponseEntity<Employee> createEmployee(@Valid @RequestBody EmployeeDto dto) {
        // If validation FAILS, this method body NEVER EVEN RUNS -
        // Spring automatically returns HTTP 400 with detailed error info BEFORE reaching here!
        Employee saved = employeeService.save(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }
}
```

The `@Valid` annotation on the `@RequestBody` parameter is what TRIGGERS validation — without it, the `@NotBlank/@Positive/etc.` annotations on the DTO are simply IGNORED entirely.

3. Key Validation Annotations

Annotation	Rule
<code>@NotNull</code>	Value must not be <code>null</code>
<code>@NotBlank</code>	String must not be <code>null</code> , empty, OR only whitespace
<code>@NotEmpty</code>	Collection/String must not be <code>null</code> or empty (but whitespace-only strings ARE allowed, unlike <code>@NotBlank</code>)
<code>@Size(min=, max=)</code>	String/Collection length must be within range
<code>@Min/@Max</code>	Numeric value must be within range
<code>@Positive/@Negative</code>	Numeric value must be positive/negative
<code>@Email</code>	Must be a validly-formatted email address
<code>@Pattern(regexp=)</code>	Must match a custom regular expression
<code>@Past/@Future</code>	Date must be in the past/future

4. Handling Validation Errors Gracefully

```
@RestControllerAdvice // covered fully in the Exception Handling topic
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidationErrors(MethodArgumentNotValidException ex)
    {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage())
        );
        return ResponseEntity.badRequest().body(errors);
        // Produces: {"firstName": "First name is required", "salary": "Salary must be positive"}
    }
}
```

5. Real Life Analogy

- **Bank:** Validation annotations are like a bank's STANDARDIZED, pre-printed form rules — a loan application form REJECTS itself automatically (before ever reaching a human loan officer) if a required field is blank or an amount is negative, exactly like `@Valid` rejecting a request BEFORE it ever reaches your controller's actual business logic.

6. Edge Cases

Case	Result
<code>@Valid</code> annotation FORGOTTEN on the <code>@RequestBody</code> parameter	ALL validation annotations on the DTO are SILENTLY IGNORED — a very common, easy-to-miss real bug
Nested objects needing validation (a DTO containing ANOTHER DTO)	Requires <code>@Valid</code> on the NESTED field too, not just the top-level parameter

7. Common Mistakes

- Forgetting `@Valid` on the controller method parameter — the validation annotations silently do NOTHING without it.
- Not providing a custom `@ExceptionHandler` for `MethodArgumentNotValidException`, resulting in a generic, unhelpful default error response instead of clear, field-specific error messages.

8. Frequently Asked Interview Questions

- 1 **What triggers Bean Validation to actually run on a `@RequestBody` DTO?** The `@Valid` annotation on that controller method parameter — without it, validation annotations on the DTO are ignored entirely.
- 2 **What's the difference between `@NotNull`, `@NotEmpty`, and `@NotBlank`?** `@NotNull` just checks non-null; `@NotEmpty` additionally requires non-empty (for Strings/Collections); `@NotBlank` additionally requires the String isn't JUST whitespace.

3 What exception is thrown when `@Valid` validation fails, and how do you handle it gracefully?

`MethodArgumentNotValidException` — typically handled via a `@RestControllerAdvice` with an `@ExceptionHandler` returning clear, field-specific error messages.

TOPIC 5: CONFIGURATION PROPERTIES

1. Introduction

What is it? Configuration Properties let you externalize application settings (database URLs, API keys, feature flags) OUTSIDE your compiled code — in `application.properties/application.yml` — and inject them cleanly into your Java classes, rather than hardcoding values directly in source code.

2. `application.properties` VS `application.yml`

```
# application.properties
server.port=8081
app.name=Employee Management System
app.max-upload-size-mb=10

# application.yml - the SAME settings, hierarchical/nested format (many prefer this for readability)
server:
  port: 8081
app:
  name: Employee Management System
  max-upload-size-mb: 10
```

3. Reading Individual Values — `@Value`

```
@Component
public class EmailService {
    @Value("${app.name}")
    private String appName;

    @Value("${app.max-upload-size-mb:5}") // ':5' is a DEFAULT value if the property isn't set
    private int maxUploadSizeMb;
}
```

4. Reading GROUPED Settings — `@ConfigurationProperties` (The Recommended, Type-Safe Way)

```
app.mail.host=smtp.gmail.com
app.mail.port=587
app.mail.from-address=noreply@company.com

@Component
@ConfigurationProperties(prefix = "app.mail")
public class MailProperties {
    private String host;
    private int port;
}
```

```

private String fromAddress;
// getters and setters REQUIRED - Hibernate-style, populated via reflection

// getters/setters...
}

@Service
public class NotificationService {
    private final MailProperties mailProperties;

    public NotificationService(MailProperties mailProperties) { // injected like any other bean
        this.mailProperties = mailProperties;
    }

    public void sendEmail() {
        System.out.println("Connecting to " + mailProperties.getHost() + ":" + mailProperties.getPort());
    }
}

```

`@ConfigurationProperties` is generally PREFERRED over scattered individual `@Value` annotations for anything with MULTIPLE related settings — it's TYPE-SAFE (compiler-checked field types), groups related config together logically, and supports validation (`@Valid` on the properties class itself).

5. Real Life Analogy

- **Restaurant:** Hardcoding a database password directly in Java source code is like a restaurant chain PERMANENTLY BAKING their supplier's secret pricing into every single printed menu — impossible to change without reprinting EVERYTHING. Configuration Properties are like keeping that information in a SEPARATE, easily-updatable master price sheet that every location's system reads from — change ONE central setting, and it applies everywhere, without touching the actual "menu" (application code) at all.

6. Edge Cases

Case	Result
A property referenced by <code>@Value</code> that doesn't exist ANYWHERE, with no default	Application FAILS to start with an error, UNLESS a default (<code>\${prop:default}</code>) is provided
<code>@ConfigurationProperties</code> class missing a setter for a property	That specific property is silently NOT bound/populated

7. Best Practices

- Use `@ConfigurationProperties` for groups of related settings; reserve `@Value` for genuinely single, standalone values.
- NEVER hardcode secrets (passwords, API keys) directly in `application.properties` committed to source control — use environment variables or a dedicated secrets manager in production (covered further in the AWS section later).

8. Frequently Asked Interview Questions

- 1 **What's the difference between `@Value` and `@ConfigurationProperties`?** `@Value` injects a SINGLE property value directly; `@ConfigurationProperties` binds an ENTIRE group of related, prefixed properties into a type-safe Java class.
 - 2 **How do you provide a default value with `@Value` if a property is missing?**
`@Value("${property.name:defaultValue}")`.
 - 3 **Why is hardcoding secrets in `application.properties` risky?** If committed to source control, secrets become permanently visible in the repository's history — production secrets should come from environment variables or a dedicated secrets manager instead.
-

TOPIC 6: PROFILES

1. Introduction

What is it? Spring Profiles let you define DIFFERENT sets of configuration/beans for DIFFERENT environments (development, testing, production) and switch between them WITHOUT changing any code — just by activating a different profile name.

2. Profile-Specific Property Files

```
application.properties <- shared defaults, ALWAYS loaded
application-dev.properties <- ONLY loaded when the "dev" profile is active
application-prod.properties <- ONLY loaded when the "prod" profile is active

# application-dev.properties
spring.datasource.url=jdbc:postgresql://localhost:5432/company_db_dev
logging.level.root=DEBUG

# application-prod.properties
spring.datasource.url=jdbc:postgresql://prod-db-server:5432/company_db
logging.level.root=WARN
```

3. Activating a Profile

```
# Via command-line argument
java -jar myapp.jar --spring.profiles.active=prod

# Via an environment variable
export SPRING_PROFILES_ACTIVE=prod

# Via application.properties itself (less common - usually set externally per environment)
spring.profiles.active=dev
```

4. Profile-Specific Beans in Code

```
@Configuration
public class PaymentConfig {

    @Bean
    @Profile("prod") // ONLY registered when the "prod" profile is active
    public PaymentGateway realPaymentGateway() {
        return new StripePaymentGateway();
    }

    @Bean
    @Profile("dev") // ONLY registered when the "dev" profile is active
    public PaymentGateway fakePaymentGateway() {
        return new FakePaymentGateway(); // simulates payments, no REAL charges during development!
    }
}
```

5. Real Life Analogy

- **Restaurant:** Profiles are like a restaurant chain having ENTIRELY different operational settings for a "TEST kitchen" (development, using fake ingredients for practice) versus their "LIVE customer-facing kitchen" (production, using real ingredients and real payment processing) — same overall recipes/code, but DIFFERENT active configuration depending on which "mode" is currently switched on.

6. Common Mistakes

- Forgetting to actually SET `spring.profiles.active` when deploying, silently running with only the base `application.properties` defaults (which may not be appropriate for that environment).
- Accidentally leaving a "dev"/"test" profile active in an actual production deployment — a genuinely dangerous real-world mistake (e.g., using fake payment gateways or debug-level logging in production).

7. Frequently Asked Interview Questions

- 1 **What are Spring Profiles used for?** Defining different configuration/beans for different environments (dev/test/prod), switchable without code changes.
- 2 **How do you activate a specific profile?** Via `--spring.profiles.active=prod` command-line argument, the `SPRING_PROFILES_ACTIVE` environment variable, or (less commonly) directly in `application.properties`.
- 3 **What does `@Profile("prod")` on a `@Bean` method do?** That bean is ONLY registered/created when the "prod" profile is currently active.

TOPIC 7: LOGGING

1. Introduction

What is it? Logging is the practice of recording application events (info messages, warnings, errors) to a log output — Spring Boot uses **SLF4J** (a LOGGING FACADE/interface) with **Logback** as the default underlying implementation, replacing ad-hoc `System.out.println()` debugging with a structured, configurable, production-grade logging system.

2. Why NOT `System.out.println()` in Production Code

Aspect	<code>System.out.println()</code>	Proper Logging (SLF4J/Logback)
Severity levels	None — everything looks the same	TRACE/DEBUG/INFO/WARN/ERROR — filterable by importance
Can be disabled/enabled per package	No	Yes — fine-grained control (e.g., DEBUG for your code, WARN for noisy libraries)
Output destination	Only the console	Console, files, remote log aggregation systems (ELK, Splunk), all configurable
Performance	Can genuinely slow down high-throughput code	Optimized, with lazy evaluation support

3. Code Example — Using SLF4J

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

@Service
public class OrderService {
    private static final Logger logger = LoggerFactory.getLogger(OrderService.class);

    public void placeOrder(Order order) {
        logger.info("Placing order for customer: {}", order.getCustomerId()); // {} = placeholder, avoids
        // string concat

        try {
            processPayment(order);
        } catch (PaymentException e) {
            logger.error("Payment failed for order {}: {}", order.getId(), e.getMessage(), e);
        }

        logger.debug("Order processing details: {}", order); // only shown if DEBUG level is enabled
    }
}
```

Using `logger.info("...{}", value)` (the placeholder pattern) instead of string concatenation (`"..." + value`) is a deliberate PERFORMANCE optimization — the string is only actually BUILT if that log level is genuinely enabled, avoiding wasted work for disabled log levels (e.g., DEBUG messages in production where only INFO+ is active).

4. Configuring Log Levels

```
# application.properties
logging.level.root=INFO # default level for EVERYTHING
logging.level.com.jobsradar.orderservice=DEBUG # MORE detailed logging for YOUR specific package
logging.level.org.hibernate.SQL=DEBUG # see the actual generated SQL (like Hibernate's show_sql!)
logging.file.name=logs/application.log # also write logs to a FILE, not just console
```

5. The Five Log Levels (Increasing Severity)

Level	When to Use
TRACE	Extremely fine-grained, rarely-needed diagnostic detail
DEBUG	Detailed information useful during development/troubleshooting
INFO	Normal, significant application events (e.g., "Order placed successfully")
WARN	Something unexpected happened, but the application can continue
ERROR	A genuine failure occurred, needing attention

6. Real Life Analogy

- **Bank:** Logging is like a bank's official transaction audit trail — every meaningful event is recorded at an appropriate SEVERITY (a routine deposit is `INFO`, a suspicious/unusual transaction pattern is `WARN`, a failed critical system process is `ERROR`) — and this trail can be reviewed later, filtered by severity, without needing to have "watched" the events happen live like an unstructured `System.out.println()` would require.

7. Common Mistakes

- Using `System.out.println()` for actual application logging instead of a proper logger — loses severity levels, filtering, and configurable output destinations.
- Logging sensitive data (passwords, full credit card numbers) — a genuine security/compliance risk.
- Using string concatenation instead of the `{}` placeholder pattern, wasting performance on string-building for log levels that aren't even enabled.

8. Frequently Asked Interview Questions

- 1 **What logging framework does Spring Boot use by default?** SLF4J as the facade/interface, with Logback as the default underlying implementation.
- 2 **Why is proper logging preferred over `System.out.println()`?** Severity levels (filterable), configurable output destinations, and better performance characteristics.
- 3 **What are the five standard log levels, from least to most severe?** `TRACE`, `DEBUG`, `INFO`, `WARN`, `ERROR`.

- 4 **Why use the `{}` placeholder pattern instead of string concatenation in log statements?** The string is only actually constructed if that log level is genuinely enabled — avoiding wasted performance on log statements that would be filtered out anyway.
-

TOPIC 8: EXCEPTION HANDLING

1. Introduction

What is it? Spring Boot provides `@RestControllerAdvice` + `@ExceptionHandler` — a CENTRALIZED mechanism for catching exceptions thrown ANYWHERE across your REST controllers and converting them into consistent, well-structured HTTP error responses, instead of scattering `try-catch` blocks throughout every individual controller method.

2. Code Example — A Global Exception Handler

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice // applies GLOBALLY, across EVERY @RestController in the application
public class GlobalExceptionHandler {

    @ExceptionHandler(EmployeeNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(EmployeeNotFoundException ex) {
        ErrorResponse error = new ErrorResponse("NOT_FOUND", ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ErrorResponse> handleBadRequest(IllegalArgumentException ex) {
        ErrorResponse error = new ErrorResponse("BAD_REQUEST", ex.getMessage());
        return ResponseEntity.badRequest().body(error);
    }

    @ExceptionHandler(Exception.class) // a CATCH-ALL fallback for anything else unhandled
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        ErrorResponse error = new ErrorResponse("INTERNAL_ERROR", "An unexpected error occurred");
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}

class ErrorResponse {
    private String code;
    private String message;
    ErrorResponse(String code, String message) { this.code = code; this.message = message; }
    // getters...
}
```

3. How Controllers Become Dramatically Cleaner

```
// WITHOUT centralized exception handling - repetitive try-catch in EVERY method
@GetMapping("/{id}")
public ResponseEntity<Employee> getEmployee(@PathVariable Long id) {
    try {
        return ResponseEntity.ok(employeeService.findById(id));
    } catch (EmployeeNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(null);
    }
}

// WITH @RestControllerAdvice - the controller just THROWS, and lets the global handler deal with it!
@GetMapping("/{id}")
public Employee getEmployee(@PathVariable Long id) {
    return employeeService.findById(id); // if this throws EmployeeNotFoundException, the GLOBAL
} // handler catches it automatically - clean, focused controller code!
```

4. Real Life Analogy

- **Bank:** `@RestControllerAdvice` is like a bank having ONE centralized "problem resolution department" that handles ALL escalated issues from EVERY branch/teller consistently — rather than every single teller needing to personally memorize and separately implement their own ad-hoc error-handling procedure.

5. Edge Cases

Case	Result
Multiple <code>@ExceptionHandler</code> methods match the SAME thrown exception's type hierarchy	Spring picks the MOST SPECIFIC matching handler (e.g., a handler for the exact exception class over one for its generic parent <code>Exception.class</code>)
An exception thrown for which NO <code>@ExceptionHandler</code> matches at all	Falls through to Spring Boot's DEFAULT error handling, typically a generic HTTP 500 response

6. Best Practices

- Define a CATCH-ALL `@ExceptionHandler(Exception.class)` as a safety net, ensuring even unanticipated errors return a CONSISTENT, well-formed error response rather than an exposed stack trace.
- Return a CONSISTENT error response STRUCTURE (like the `ErrorResponse` class above) across ALL your exception handlers, so API consumers can reliably parse errors.
- NEVER expose internal exception details/stack traces directly to API consumers in production — map to a clean, generic message instead.

7. Frequently Asked Interview Questions

- 1 **What does `@RestControllerAdvice` do?** Applies exception handling GLOBALLY, across every `@RestController` in the application, rather than requiring repetitive try-catch blocks in each controller method.

- 2 **What does `@ExceptionHandler` specify?** Which specific exception TYPE a particular handler method should catch and convert into an HTTP response.
 - 3 **What happens if multiple handler methods could match a thrown exception's type hierarchy?** Spring selects the MOST SPECIFIC matching handler.
 - 4 **Why should you include a catch-all `ExceptionHandler` handler?** As a safety net, ensuring even unanticipated errors return a clean, consistent error response instead of an exposed internal stack trace.
-
-

TOPIC 9: SCHEDULING

1. Introduction

What is it? Spring's `@Scheduled` annotation lets you run a method AUTOMATICALLY at fixed intervals or specific times (e.g., "run this every night at 2 AM," "run this every 30 seconds") — WITHOUT manually managing a `Timer`/`Thread` yourself.

2. Enabling Scheduling

```
@SpringBootApplication
@EnableScheduling // MUST be added to enable @Scheduled methods anywhere in the application
public class MyApplication { }
```

3. Code Examples — The Three Scheduling Styles

```
@Component
public class ReportScheduler {

    @Scheduled(fixedRate = 60000) // runs every 60,000ms (60 sec), measured from the START of each run
    public void generateHourlyReport() {
        System.out.println("Generating report at: " + LocalDateTime.now());
    }

    @Scheduled(fixedDelay = 30000) // runs 30 sec AFTER the PREVIOUS execution FINISHES
    public void syncData() {
        System.out.println("Syncing data...");
    }

    @Scheduled(cron = "0 0 2 * * *") // CRON expression: every day at 2:00 AM
    public void nightlyCleanup() {
        System.out.println("Running nightly cleanup...");
    }
}
```

4. `fixedRate` VS `fixedDelay` — A Frequently Confused Pair

Aspect	<code>fixedRate</code>	<code>fixedDelay</code>
Timing measured from	The START of the PREVIOUS execution	The END (completion) of the PREVIOUS execution
Risk	If a task takes LONGER than the rate, executions can start OVERLAPPING/queuing up	Guarantees a genuine GAP between the end of one run and the start of the next — never overlaps

5. Cron Expression Format

```
"0 0 2 * * *"  
| | | | |  
| | | | +-- day of week (* = every day)  
| | | +---- month (* = every month)  
| | +----- day of month (* = every day)  
| +----- hour (2 = 2 AM)  
| +----- minute (0)  
+----- second (0)
```

6. Real Life Analogy

- **Bank:** `@Scheduled` is like a bank's automated end-of-day batch processing — interest calculations run automatically every night at a fixed time, with no human needing to manually trigger the process; the bank simply "sets and forgets" the schedule.

7. Edge Cases

Case	Result
A <code>fixedRate</code> task that takes LONGER to run than the specified rate	By default, Spring's SINGLE-threaded scheduler queues the next execution, potentially causing a backlog if this happens repeatedly — a genuine production concern for long-running scheduled tasks
Multiple <code>@Scheduled</code> methods across the application, by default	ALL share a SINGLE background thread by default — one SLOW scheduled task can DELAY others; a dedicated thread pool (<code>@Async</code> , next topic, or a configured <code>TaskScheduler</code>) is needed for genuine concurrency between scheduled tasks

8. Common Mistakes

- Not realizing ALL `@Scheduled` methods share ONE thread by default — a single slow/stuck task can silently delay every OTHER scheduled task in the application.
- Confusing `fixedRate` (measured from start) with `fixedDelay` (measured from completion), leading to unexpected overlapping or gapped execution.

9. Frequently Asked Interview Questions

- 1 **What annotation enables Spring's scheduling feature?** `@EnableScheduling` on a configuration/main application class.
 - 2 **What's the difference between `fixedRate` and `fixedDelay`?** `fixedRate` measures the interval from the START of the previous execution (risking overlap if a task runs long); `fixedDelay` measures from the END of the previous execution (guaranteeing a genuine gap).
 - 3 **How do you schedule a task to run at a specific time daily (e.g., 2 AM)?** Using a cron expression: `@Scheduled(cron = "0 0 2 * * *")`.
 - 4 **Do multiple `@Scheduled` methods run concurrently by default?** No — they share a single background thread by default; a slow task can delay others unless a dedicated thread pool is configured.
-

TOPIC 10: ASYNC PROGRAMMING

1. Introduction

What is it? Spring's `@Async` annotation lets you run a method on a SEPARATE background thread, RETURNING CONTROL to the caller IMMEDIATELY, rather than blocking the calling thread until that method completes — useful for long-running operations (sending an email, generating a report) that don't need to hold up the main request/response flow.

2. Enabling and Using `@Async`

```
@SpringBootApplication
@EnableAsync // MUST be added to enable @Async methods
public class MyApplication { }

@Service
public class EmailService {

    @Async
    public void sendWelcomeEmail(String email) {
        // simulate a SLOW operation (e.g., a real network call to an email provider)
        try { Thread.sleep(3000); } catch (InterruptedException e) { }
        System.out.println("Welcome email sent to: " + email + " on thread: " +
            Thread.currentThread().getName());
    }
}

@RestController
public class SignupController {
    private final EmailService emailService;

    public SignupController(EmailService emailService) { this.emailService = emailService; }

    @PostMapping("/signup")
    public String signup(@RequestBody SignupRequest request) {
        // ... save the user to the database (FAST, synchronous) ...
        emailService.sendWelcomeEmail(request.getEmail()); // returns IMMEDIATELY - doesn't wait 3 seconds!
        return "Signup successful!"; // the HTTP RESPONSE returns to the user RIGHT AWAY
    }
}
```

```
}
```

Without `@Async`, the `signup()` method would BLOCK for the full 3 seconds waiting for the email to send BEFORE returning a response to the user — a poor user experience for something the user doesn't need to wait for synchronously.

3. Getting a Return Value from an Async Method — `CompletableFuture`

```
@Async
public CompletableFuture<String> generateReport() {
    // ... slow report generation logic ...
    return CompletableFuture.completedFuture("Report data here");
}
```

(This ties directly back to `[[CompletableFuture]]` from Core Java's Multithreading section — Spring's `@Async` is essentially a convenient, annotation-driven wrapper around submitting work to a background thread pool.)

4. Real Life Analogy

- **Restaurant:** `@Async` is like a waiter taking a customer's SPECIAL, slow-to-prepare dessert order, handing it to the kitchen to prepare IN THE BACKGROUND, and IMMEDIATELY moving on to serve other tables — rather than the waiter standing frozen at that one table for the entire dessert preparation time before doing anything else.

5. Edge Cases

Case	Result
Calling an <code>@Async</code> method from WITHIN the SAME class (<code>this.asyncMethod()</code>)	Does NOT actually run asynchronously! Exactly the SAME limitation as AOP (Spring Core, Topic 10) — Spring's proxy-based mechanism only intercepts calls from OUTSIDE the class
An exception thrown inside an <code>@Async</code> void method	By default, SILENTLY LOGGED but not propagated back to the caller (since the caller already moved on) — requires a custom <code>AsyncUncaughtExceptionHandler</code> to handle deliberately

6. Common Mistakes

- Calling an `@Async` method internally (`this.method()`) within the same class and being confused why it still runs synchronously — the SAME proxy limitation as AOP.
- Not configuring a proper, appropriately-sized thread pool for `@Async` tasks, relying on Spring's small default pool, which can become a bottleneck under real load.

7. Frequently Asked Interview Questions

- 1 **What does `@Async` do?** Runs a method on a separate background thread, allowing the caller to continue immediately without waiting for that method to complete.

- 2 **What annotation enables `@Async` support?** `@EnableAsync` on a configuration/main application class.
 - 3 **Why doesn't calling an `@Async` method from within the SAME class actually run asynchronously?** Spring's proxy-based mechanism only intercepts calls made from OUTSIDE the class — the exact same limitation as Spring AOP.
 - 4 **How do you get a return value back from an `@Async` method?** Have it return a `CompletableFuture<T>` (or `Future<T>`), which the caller can later `.get()`/chain against.
-

TOPIC 11: FILE UPLOAD

1. Introduction

What is it? Spring Boot's `MultipartFile` interface provides a straightforward way to receive uploaded files (images, documents) via an HTTP request, without manually parsing the raw multipart/form-data HTTP protocol yourself.

2. Code Example — A File Upload Endpoint

```
import org.springframework.web.multipart.MultipartFile;
import org.springframework.web.bind.annotation.*;
import java.io.IOException;
import java.nio.file.*;

@RestController
@RequestMapping("/api/files")
public class FileUploadController {

    private static final String UPLOAD_DIR = "uploads/";

    @PostMapping("/upload")
    public ResponseEntity<String> uploadFile(@RequestParam("file") MultipartFile file) {
        if (file.isEmpty()) {
            return ResponseEntity.badRequest().body("File is empty");
        }

        try {
            Path uploadPath = Paths.get(UPLOAD_DIR);
            if (!Files.exists(uploadPath)) {
                Files.createDirectories(uploadPath);
            }
            Path filePath = uploadPath.resolve(file.getOriginalFilename());
            Files.copy(file.getInputStream(), filePath, StandardCopyOption.REPLACE_EXISTING);

            return ResponseEntity.ok("File uploaded successfully: " + file.getOriginalFilename());
        } catch (IOException e) {
            return ResponseEntity.status(500).body("Upload failed: " + e.getMessage());
        }
    }
}
```

3. Configuring Upload Size Limits

```
# application.properties
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
```

4. `MultipartFile` Key Methods

Method	Purpose
<code>getOriginalFilename()</code>	The file's original name as uploaded by the client
<code>getSize()</code>	The file's size in bytes
<code>getContentType()</code>	The MIME type (e.g., "image/png")
<code>getInputStream()</code>	An <code>InputStream</code> to read the file's actual content
<code>isEmpty()</code>	Checks if the "file" is actually empty (a common defensive check)

5. Real Life Analogy

- **Restaurant:** File upload handling is like a restaurant's dedicated intake counter for customer-submitted documents (catering order forms, special dietary requirement letters) — a standardized process for RECEIVING and STORING an arbitrary file, checking its size/type before accepting it, exactly like validating an uploaded file's size and content type.

6. Edge Cases

Case	Result
An uploaded file EXCEEDING <code>max-file-size</code>	<code>MaxUploadSizeExceededException</code> — should be handled via <code>@ExceptionHandler</code> (Topic 8) for a clean error response
No filename in the uploaded file (empty string)	<code>getOriginalFilename()</code> returns an empty string — should be validated before using it to construct a file path
Using the CLIENT-PROVIDED filename directly, unsanitized, as the server-side storage path	A serious security risk (PATH TRAVERSAL attack, e.g., " <code>../../etc/passwd</code> ") — always sanitize/generate a SAFE filename server-side

7. Common Mistakes

- Using the client-supplied original filename directly as the storage path without sanitization — a genuine path traversal security vulnerability.
- Not setting explicit `max-file-size`/`max-request-size` limits, risking resource exhaustion from unexpectedly huge uploads.

8. Best Practices

- Generate your OWN safe, unique filename server-side (e.g., a UUID) rather than trusting the client-provided filename directly.
- Always validate file size AND content type before processing/storing an uploaded file.
- Store uploaded files outside the application's own deployable directory structure (or in dedicated cloud storage like AWS S3, covered later in this course) rather than directly alongside application code.

9. Frequently Asked Interview Questions

- 1 **What Spring interface represents an uploaded file?** `MultipartFile`.
- 2 **Why is using the client's original filename directly as a server-side storage path dangerous?** It risks a path traversal attack, where a maliciously crafted filename (like `"../../etc/passwd"`) could let an attacker write to unintended locations — always sanitize or generate your own safe filename instead.
- 3 **How do you configure maximum upload file size in Spring Boot?** Via `spring.servlet.multipart.max-file-size` (and `max-request-size`) in `application.properties`.

TOPIC 12: EMAIL INTEGRATION

1. Introduction

What is it? Spring Boot's `spring-boot-starter-mail` provides a simple, standardized way to send emails from your application (welcome emails, password resets, notifications) via the `JavaMailSender` interface.

2. Configuration

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-mail</artifactId>
</dependency>

spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=your-email@gmail.com
spring.mail.password=your-app-specific-password
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

3. Code Example — Sending a Simple Email

```
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;

@Service
public class EmailService {

    private final JavaMailSender mailSender;

    public EmailService(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    @Async // combine with Topic 10 - don't block the caller waiting on SMTP!
    public void sendWelcomeEmail(String toEmail, String userName) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setTo(toEmail);
        message.setSubject("Welcome to our platform!");
        message.setText("Hi " + userName + ", thanks for signing up!");
        mailSender.send(message);
    }
}
```

4. HTML Emails with Attachments (`MimeMessage`)

```
import jakarta.mail.internet.MimeMessage;
import org.springframework.mail.javamail.MimeMessageHelper;

public void sendHtmlEmailWithAttachment(String toEmail) throws Exception {
    MimeMessage mimeMessage = mailSender.createMimeMessage();
    MimeMessageHelper helper = new MimeMessageHelper(mimeMessage, true); // 'true' enables multipart (for attachments)

    helper.setTo(toEmail);
    helper.setSubject("Your Invoice");
    helper.setText("<h1>Thank you for your order!</h1><p>Please find your invoice attached.</p>", true);
    // 'true' = HTML
    helper.addAttachment("invoice.pdf", new File("invoice.pdf"));

    mailSender.send(mimeMessage);
}
```

5. Real Life Analogy

- **Bank:** `JavaMailSender` is like a bank's standardized outgoing-mail department — any part of the bank's system can hand it a properly formatted notice (a message object), and it handles ALL the actual mail-delivery mechanics (connecting to the postal service/SMTP server) uniformly, without each individual department needing to know those low-level delivery details itself.

6. Common Mistakes

- Sending emails SYNCHRONOUSLY inside a user-facing request flow (e.g., during signup) without `@Async` — makes the user wait unnecessarily for a slow external SMTP round-trip.

- Hardcoding real email credentials directly in `application.properties` committed to source control — use environment variables/secrets management instead (same principle as Topic 5).

7. Frequently Asked Interview Questions

- 1 **What Spring interface is used to send emails?** `JavaMailSender`.
- 2 **Why is it a good idea to combine email sending with `@Async`?** Sending an email involves a slow external network call (SMTP) — running it asynchronously avoids blocking the user-facing request that triggered it.
- 3 **How do you send an HTML email with an attachment?** Use `MimeMessageHelper` with `MimeMessage`, setting `setText(html, true)` for HTML content and `addAttachment()` for files.

TOPIC 13: SWAGGER/OPENAPI

1. Introduction

What is it? Swagger/OpenAPI is a specification (and a set of tools) for DOCUMENTING REST APIs automatically — generating an interactive, browsable web page listing every endpoint, its parameters, request/response formats, and even letting you TEST calls directly from the browser, all AUTOMATICALLY GENERATED from your actual Spring Boot controller code.

2. Why It Matters — The Problem It Solves

Without automated API documentation, teams typically rely on MANUALLY WRITTEN documentation (a Word doc, a wiki page) describing available endpoints — which INEVITABLY drifts out of sync with the actual code as the API evolves over time. Swagger/OpenAPI generates documentation DIRECTLY from your live code/annotations, guaranteeing it's ALWAYS accurate and up to date.

3. Setup

```
<dependency>
<groupId>org.springdoc</groupId>
<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
<version>2.3.0</version>
</dependency>
```

Once added, simply run your application and visit `http://localhost:8080/swagger-ui.html` — a FULL, INTERACTIVE API documentation page is generated AUTOMATICALLY, with zero additional code required, just from your existing `@RestController` classes.

4. Enhancing Documentation with Annotations

```
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.tags.Tag;

@RestController
@RequestMapping("/api/employees")
@Tag(name = "Employee Management", description = "APIs for managing employees")
public class EmployeeController {

    @Operation(summary = "Get an employee by ID", description = "Returns a single employee's full details")
    @ApiResponse(responseCode = "200", description = "Employee found")
    @ApiResponse(responseCode = "404", description = "Employee not found")
    @GetMapping("/{id}")
    public Employee getEmployee(@PathVariable Long id) {
        return employeeService.findById(id);
    }
}
```

5. Real Life Analogy

- **Restaurant:** Swagger/OpenAPI is like a restaurant's menu being AUTOMATICALLY GENERATED and kept perfectly up to date directly from the kitchen's actual current recipe database — instead of a separately, manually-typed paper menu that inevitably falls out of sync whenever a dish changes, guests ALWAYS see an accurate, interactive, browsable menu reflecting exactly what the kitchen can currently prepare.

6. Common Mistakes

- Relying on a separate, manually-maintained document/wiki for API documentation instead of Swagger/OpenAPI — inevitably goes stale as the code evolves.
- Leaving Swagger UI PUBLICLY accessible in a production deployment without any access restriction — can expose internal API structure/details unnecessarily; often disabled or secured in production environments.

7. Frequently Asked Interview Questions

- 1 **What is Swagger/OpenAPI used for?** Automatically generating interactive, always-up-to-date REST API documentation directly from your Spring Boot controller code.
- 2 **What's the main advantage of Swagger over manually-written API documentation?** It's generated directly from the live code, guaranteeing it never drifts out of sync as the API evolves — manual documentation inevitably becomes stale.
- 3 **What should you consider about Swagger UI in a production environment?** Whether it should be disabled or access-restricted, since it can expose your internal API structure publicly if left openly accessible.

8. Revision Notes (Entire Spring Boot Section Consolidated)

SPRING BOOT – CHEAT SHEET

=====

AUTO CONFIGURATION: auto-configures beans based on classpath + properties, using

@ConditionalOnClass/@ConditionalOnMissingBean - your OWN @Bean always wins

STARTER DEPENDENCIES: bundles compatible library groups (spring-boot-starter-web, -data-jpa, ...)

REST APIs: @RestController + @GetMapping/@PostMapping/etc. + @PathVariable/@RequestParam/

@RequestBody + ResponseEntity for full status-code control

VALIDATION: @Valid on @RequestBody + @NotNull/@NotBlank/@Size/@Email/etc. on the DTO

- @Valid MUST be present or validation silently does nothing

CONFIGURATION PROPERTIES: @Value (single) vs @ConfigurationProperties (grouped, type-safe)

PROFILES: application-{profile}.properties + @Profile("name") on beans, activate via

--spring.profiles.active=prod

LOGGING: SLF4J + Logback, 5 levels (TRACE<DEBUG<INFO<WARN<ERROR), use {} placeholders

EXCEPTION HANDLING: @RestControllerAdvice + @ExceptionHandler - centralized, clean controllers

SCHEDULING: @EnableScheduling + @Scheduled(fixedRate=/fixedDelay=/cron=)

fixedRate = from START of previous run; fixedDelay = from END of previous run

ASYNC: @EnableAsync + @Async - runs on a background thread, doesn't block the caller

GOTCHA: internal this.method() calls do NOT go through the proxy - stay synchronous!

FILE UPLOAD: MultipartFile - NEVER trust the client's filename directly (path traversal risk)

EMAIL: JavaMailSender + SimpleMailMessage/MimeMessageHelper - combine with @Async

SWAGGER/OPENAPI: auto-generated, always-accurate interactive API docs from your controllers

9. Homework (Covers Entire Spring Boot Section)

- 1 Build a complete `Employee` REST API (GET all, GET by id, POST, PUT, DELETE) with a validated DTO, backed by an in-memory list (or your PostgreSQL database from earlier sections).
- 2 Add a `@RestControllerAdvice` handling both `EmployeeNotFoundException` and validation errors with clean, structured JSON responses.
- 3 Add a `@Scheduled` task that logs "Health check OK" every 30 seconds, and an `@Async` method that simulates a slow email send without blocking a REST endpoint.
- 4 Add Swagger/OpenAPI to your project and browse your own generated API documentation at `/swagger-ui.html`.

10. Mini Project — A Complete Employee Management REST API

```
package com.jobsradar.employeeapi;

import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

record EmployeeDto(
    @NotBlank(message = "Name is required") String name,
    @NotNull @Positive(message = "Salary must be positive") Double salary
) {}

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private static final Logger logger = LoggerFactory.getLogger(EmployeeController.class);
    private final Map<Long, EmployeeDto> employees = new ConcurrentHashMap<>();
    private final AtomicLong idGenerator = new AtomicLong(1);

    @GetMapping
    public Collection<EmployeeDto> getAll() {
        return employees.values();
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public EmployeeDto create(@Valid @RequestBody EmployeeDto dto) {
        long id = idGenerator.getAndIncrement();
        employees.put(id, dto);
        logger.info("Created employee with id {}: {}", id, dto.name());
        return dto;
    }

    @GetMapping("/{id}")
    public ResponseEntity<EmployeeDto> getById(@PathVariable Long id) {
        EmployeeDto emp = employees.get(id);
        return emp != null ? ResponseEntity.ok(emp) : ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        employees.remove(id);
        return ResponseEntity.noContent().build();
    }
}
```

FINAL SUMMARY — SPRING BOOT COMPLETE

This concludes the **Spring Boot** section — 13 topics:

- 1 **Auto Configuration** — the exact boilerplate problem it eliminates, `@ConditionalOnClass/@ConditionalOnMissingBean`.
- 2 **Starter Dependencies** — bundled, version-managed library groups.
- 3 **REST APIs** — full CRUD controller implementation with every key annotation.
- 4 **Validation** — `@Valid` + Bean Validation annotations, the "forgot `@Valid`" gotcha.
- 5 **Configuration Properties** — `@Value` vs `@ConfigurationProperties`.
- 6 **Profiles** — environment-specific configuration and beans.
- 7 **Logging** — SLF4J/Logback, log levels, why not `System.out.println()`.
- 8 **Exception Handling** — `@RestControllerAdvice` for centralized, clean error handling.
- 9 **Scheduling** — `@Scheduled`, `fixedRate` vs `fixedDelay`, cron expressions.
- 10 **Async Programming** — `@Async`, the same internal-call proxy limitation as AOP.
- 11 **File Upload** — `MultipartFile`, the path traversal security trap.
- 12 **Email Integration** — `JavaMailSender`, combining with `@Async`.
- 13 **Swagger/OpenAPI** — always-accurate, auto-generated API documentation.

This is the most practically job-relevant section so far — everything here is what you'll actually write day-to-day in a real Spring Boot role. Next: Spring Data JPA, which builds an even higher-level, more convenient repository API on top of the Hibernate concepts you already know.

(END OF SPRING BOOT MASTER NOTES — ready to proceed to Spring Data JPA whenever you say "Next Topic.")